Подготовка к лабораторной работе № 4

Учебный конспект: Windows Script Host, VBScript/JScript и автоматизация приложений

Материал для самостоятельного изучения и подготовки к защите. Рассчитан на новичка с Windows 11. Примеры связаны с вашей лабораторной; выполнять их на ноутбуке следует в обычном пользовательском сеансе, по одному.

1. Что нужно уметь объяснить

После изучения вы должны своими словами объяснять, кто выполняет .vbs и .js, какие объекты создаются, чем запуск программы отличается от активации её окна, как устроен SendKeys, как читаются файлы, как выбирается система счисления, зачем нужен парсер и почему сбой буфера обмена не равен арифметической ошибке.

Порядок подготовки: сначала разделы 2–6, затем системы счисления и файлы, после этого разбор третьего задания. В конце выполните упражнения и ответьте на вопросы без подсказки. Если ответ не получается, возвращайтесь к разделу, а не заучивайте только формулировку.

2. Картина всей работы

Есть сценарий, интерпретатор и управляемые программы. Сценарий — текст с инструкциями. Интерпретатор VBScript или JScript понимает эти инструкции. Хост WSH предоставляет окружение для их выполнения. Через COM-объект оболочки сценарий запускает приложения и обращается к их окнам. Операционная система управляет процессами, окнами, файловой системой и буфером обмена.

В третьем задании маршрут такой: строка файла → проверка синтаксиса → перевод входных чисел во внутренние значения → дерево операций → запросы Калькулятору → перевод ответа в выбранное основание → выходной файл.

Не смешивайте четыре вещи: язык JScript, хост cscript.exe, библиотеку Scripting.FileSystemObject и приложение Калькулятор. Они выполняют разные роли.

3. Файлы, пути и командная строка Windows

Полный путь содержит диск и каталог: C:\STUDENT\ИИ-551\Lab4\script3.js. Относительный путь text1.txt зависит от текущего каталога. В коде путь через WScript.ScriptFullName привязывает поиск к месту самого сценария и не зависит от того, откуда его запустили.

```
mkdir "C:\STUDENT\ИИ-551\Lab4"
cd /d "C:\STUDENT\ИИ-551\Lab4"
dir
notepad text1.txt
cscript.exe //nologo //E:JScript script3.js
```

Команда	Как прочитать и что произойдёт
mkdir "каталог"	Создать папку для файлов лабораторной.
cd /d "каталог"	Сменить текущий диск и папку. Без /d смена диска может не произойти.

Команда	Как прочитать и что произойдёт
dir	Посмотреть список файлов; это первый шаг при ошибке «Файл не найден».
notepad имя	Открыть указанный файл в Блокноте.
cscript.exe //nologo script1.vbs	Выполнить VBScript в консоли, без баннера.
cscript.exe //nologo //E:JScript script3.js	Выполнить через JScript явно; не использовать браузерную ассоциацию .js.
wscript.exe script1.vbs	Выполнить через оконный хост.
echo %errorlevel%	В cmd сразу после запуска показать код завершения; промежуточные команды могут его изменить.
where cscript	Найти cscript.exe по системному PATH. Это поиск программы, а не проверка корректности сценария.

Кавычки защищают путь с пробелами. Расширение важно: script1.vbs.txt — текстовый файл, не сценарий .vbs. Включите «Проводник → Вид → Показать → Расширения имён файлов». Не запускайте файлы прямо из ZIP: сначала распакуйте весь комплект.

4. WScript, Shell и COM

Объект	Откуда берётся	Для чего нужен
WScript	Предоставляется WSH автоматически.	Sleep, Echo, Quit, ScriptFullName, Arguments.
WScript.Shell	CreateObject или ActiveXObject.	Run, Exec, AppActivate, SendKeys, Popup.
Scripting.FileSystemObject	CreateObject или ActiveXObject.	Файлы, папки и текстовые потоки.

COM — механизм Windows, позволяющий программам обращаться к объектам с определёнными методами и свойствами. Для защиты достаточно понимать: сценарий получает объект, а затем вызывает его методы. Не всякое приложение предоставляет COM-автоматизацию; запуск calc.exe через Run не означает, что у Калькулятора есть доступный объект Calculator.Application.

```
Option Explicit
Dim sh
Set sh = CreateObject("WScript.Shell")
WScript.Echo "Объект оболочки создан"

var sh = new ActiveXObject("WScript.Shell");
WScript.Echo("Объект оболочки создан");
```

В VBScript Set присваивает ссылку на объект. В JScript используется обычное присваивание. Метод — действие, например Run(...). Свойство — значение, например WScript.ScriptFullName. Не путайте строку "WScript.Shell" с именем переменной sh: переменную можно назвать иначе, COM-имя объекта должно оставаться правильным.

5. Минимум VBScript для чтения первых двух сценариев

Синтаксис	Пояснение
Option Explicit	Запрет использования необъявленных переменных.
Dim sh, i	Объявление переменных. Тип определяется во время работы.
Set sh = ...	Присвоение объектной ссылки.
' комментарий	Текст до конца строки не исполняется.
If условие Then ... End If	Условное выполнение.
For i = 1 To 30 ... Next	Цикл с 30 значениями счётчика.
Sub Имя(параметр) ... End Sub	Процедура без возвращаемого значения.
Function Имя(...) ... End Function	Функция; результат присваивается её имени.
Exit Sub	Выйти только из процедуры.
WScript.Quit 1	Завершить весь сценарий с кодом 1.
a & b	Соединить строки. Не путать с логическим And.
vbCrLf	Перевод строки Windows: CR и LF.

Вызов без использования результата в VBScript часто записывается без скобок: sh.Run "calc.exe", 1, False. Если результат сохраняется, используется форма result = sh.Run("...", 1, True). Не переносите механически синтаксис вызова из JScript в VBScript.

6. Запуск и ожидание: Run и Exec

```
sh.Run "calc.exe", 1, False
WScript.Sleep 2000
```

Run(command, windowStyle, wait) запускает команду. Первый аргумент — командная строка; второй — способ отображения; третий — ждать ли завершения. False удобно для Калькулятора: сценарий продолжает работать, пока приложение открыто. Если поставить True для обычного запуска Калькулятора, сценарий может ждать его закрытия вместо передачи клавиш.

Run с ожиданием возвращает код завершения. Значение Run без ожидания не является PID. Exec возвращает объект процесса с ProcessID, Status, ExitCode и потоками StdIn, StdOut, StdErr. Exec полезен для консольных утилит; он не заменяет механизм управления GUI.

```
var sh = new ActiveXObject("WScript.Shell");
var command = sh.ExpandEnvironmentStrings("%COMSPEC%") + " /c echo LAB4";
var process = sh.Exec(command);
WScript.Echo(process.StdOut.ReadAll());
while (process.Status === 0) WScript.Sleep(100);
WScript.Echo("ExitCode: " + process.ExitCode);
```

Это маленький учебный вывод, не шаблон для больших потоков: при большом объёме вывода нельзя бесконтрольно ждать завершения, не читая stdout/stderr, иначе возможна блокировка заполненных буферов. %COMSPEC% — путь к командному интерпретатору, /c — выполнить команду и завершиться.

WScript.Sleep измеряется в миллисекундах: 500 = 0.5 с, 2000 = 2 с. Пауза нужна, потому что интерфейс реагирует не мгновенно. Ожидание условия с ограничением времени надёжнее одной фиксированной паузы.

7. Процесс, окно и AppActivate

PID — идентификатор процесса, HWND — дескриптор окна. В учебном разговоре нельзя называть их одним и тем же числом. Одно приложение может иметь несколько процессов и окон.

AppActivate("Калькулятор") пытается найти окно по заголовку и активировать его. При успехе возвращает True. Метод не выполняет арифметику, не запускает отсутствующее приложение и не гарантирует, что пользователь не сменит фокус через мгновение.

Алгоритм ActivateCalc: до 30 попыток → русское/английское имя → при успехе пауза и выход → иначе пауза и повтор → после лимита сообщение и Quit(1). При нескольких окнах с одинаковыми заголовками выбор может оказаться неоднозначным.

В исходной версии VBScript выражение A Or B вычисляет обе части: короткого замыкания нет. Поэтому даже успешная первая активация не исключает второго вызова. Вариант If ... ElseIf ... позволяет выполнять проверки последовательно. В JScript логические || и && обычно используют короткое замыкание.

8. SendKeys: шпаргалка

SendKeys отправляет имитацию клавиатурного ввода активному окну. Он не принимает произвольный текст как надёжную независимую от раскладки запись. Для русских подписей предпочтительнее Unicode-файл или другой прямой механизм передачи текста.

Строка	Интерпретация
9*6=	Ввод выражения в Калькулятор.
^c	Ctrl+C. Результат копирования определяется активным приложением.
^v	Ctrl+V.
%1 / %3	Alt+1 / Alt+3. В используемом Калькуляторе — обычный режим / программист.
+a	Shift+A.
{+}	Буквальный знак +, а не модификатор Shift.
{ENTER}	Клавиша Enter.
{ESC}	Escape; в Калькуляторе очищает текущее состояние в используемом режиме.
{TAB}	Tab, переход фокуса.
{F5} / {F6}	Функциональные F5 / F6; HEX / DEC в режиме программиста.
{F9}	Смена знака в используемом Калькуляторе.
F7	Символы F и 7, число $F7_{16}$ при HEX-вводе.

Строка	Интерпретация
{F7}	Функциональная клавиша; ОСТ в режиме программиста.
{LEFT}, {RIGHT}, {HOME}, {END}	Навигационные клавиши.

Спецсимволы +, ^, %, ~, (,), {, } требуют особой записи, если должны вводиться буквально. Например, круглые скобки группируют клавиши под модификатором; для буквальных скобок применяются {(} и {)}. Для + корректный пример в лабораторной — F7{+}7F=.

Частая ошибка: послать ^c, когда активен Блокнот, а не Калькулятор. Ещё одна — переключить окно мышью во время сценария. Правильное число в Калькуляторе при пустом выходе обычно направляет поиск ошибки к фокусу и обмену, а не к умножению.

9. Диалоги: Popup и MsgBox

Popup принадлежит WScript.Shell и доступен из обоих языков. MsgBox — встроенная функция VBScript. В Popup указываются текст, тайм-аут, заголовок и тип окна.

```
var sh = new ActiveXObject("WScript.Shell");
var answer = sh.Popup("Продолжить?", 0, "Учебный пример", 3 + 32);
WScript.Echo("Код выбранной кнопки: " + answer);
```

Число	Значение в Popup
0 в параметре тайм-аута	Ждать ответа без ограничения.
3 в параметре типа	Кнопки Да / Нет / Отмена.
32 в параметре типа	Значок вопроса; тип 3 + 32 = 35.
1 / 2 / 6 / 7 в результате	OK / Отмена / Да / Нет.
-1 в результате	Истёк заданный ненулевой тайм-аут.

Для выбора четырёх оснований одного стандартного диалога с тремя кнопками недостаточно. Поэтому сценарий задаёт последовательность вопросов. Важно проследить все ветви: в старом чужом образце вариант DEC оказался недоступен, хотя текст диалога его обещал.

10. FileSystemObject и кодировки

```
var fso = new ActiveXObject("Scripting.FileSystemObject");
var dir = fso.GetParentFolderName(WScript.ScriptFullName);
var file = fso.OpenTextFile(dir + "\\text1.txt", 1, false, 0);
while (!file.AtEndOfStream) WScript.Echo(file.ReadLine());
file.Close();
```

В строковом литерале JScript обратная косая черта экранируется: "\\" означает одну косую черту в фактическом пути. В VBScript обратная косая черта не требует такого удвоения.

Метод / значение	Назначение
FileExists(path)	Есть ли файл. Правильное имя во множественном числе: Exists.
FolderExists(path)	Есть ли каталог.

Метод / значение	Назначение
BuildPath(a,b)	Составить путь из каталога и имени.
GetParentFolderName(path)	Получить родительский каталог.
OpenTextFile(path, 1, false, 0)	Чтение существующего файла в ANSI-режиме.
Режимы 1 / 2 / 8	Чтение / запись / добавление в конец.
CreateTextFile(path, true, true)	Перезаписать или создать Unicode UTF-16LE-файл.
ReadLine / ReadAll	Прочитать строку / весь оставшийся текст.
AtEndOfStream	Достигнут ли конец потока.
Write / WriteLine	Записать текст без / с переводом строки.
Close	Закрыть поток, освободить ресурс.
DeleteFile(path)	Удалить файл. В лабораторной — собственные временные запросы/ответы.
GetSpecialFolder(2)	Системный временный каталог.
GetTempName()	Получить имя для временного файла; само содержимое создаётся отдельно.

Для OpenTextFile формат 0 — ASCII/ANSI в терминологии FSO, -1 — Unicode UTF-16, -2 — системный формат по умолчанию. Это не универсальный UTF-8-декодер. true в третьем аргументе CreateTextFile означает Unicode, а true во втором — перезапись; одинаковое слово занимает разные позиции и имеет разный смысл.

ASCII — общий набор латинских символов и цифр. UTF-8 и UTF-16 — разные кодировки Unicode. BOM — сигнатура в начале файла. Текущий входной reader не удаляет UTF-8 BOM; первый символ тогда может вызвать ошибку. Для лабораторной проще сохранять ASCII-содержимое без BOM. Перезаписываемые результаты следует копировать под другим именем перед следующим запуском.

11. Минимум JScript для чтения третьего задания

Элемент	Смысл
var pos = 0;	Объявление и инициализация переменной.
function name(a) { ... }	Объявление функции.
return value;	Возврат значения из функции.
=== / !==	Строгое сравнение без автоматического приведения типов.
if / else	Ветвление.
for / while	Циклы.
[] / push	Массив / добавить элемент в конец.
{kind: "number", value: n}	Объект с полями; так хранится узел дерева.
try / catch	Перехват исключения и обработка ошибки.
throw new Error(text)	Создать и передать ошибку вызывающему коду.
condition ? a : b	Выбрать одно из двух значений по условию.
Number(s)	Преобразовать строку числового ответа в Number.
isFinite(n)	Проверить конечность числа; помогает отсеять Infinity и NaN после преобразования.
charAt / substr / split	Символ строки / подстрока / разделение по разделителю.
replace / toUpperCase	Заменить совпадения / перевести буквы в верхний регистр.
Math.floor / Math.abs	Округление вниз / модуль.

WSH JScript — старый движок. Не следует без проверки использовать let, const, стрелочные функции, шаблонные строки и String.trim. В приведённом сценарии используются var, обычные функции и регулярные выражения. Современный JavaScript Node.js может выполнять тестовую часть ядра, но не заменяет COM-объекты Windows.

12. Регулярные выражения из программы

Выражение	Что проверяет или меняет
/\s/	Один пробельный символ.
/^\s*$/	Вся строка состоит из нуля или более пробельных символов. ^ и $ — границы строки.
/[0-9]/	Одна десятичная цифра.
/[0-9a-fA-F.]/	Символ-кандидат для числа: цифра, A–F или точка. Дальше нужна проверка выбранного основания.
/0+$/	Один или более нулей в конце строки; удаляет завершающие нули дробной части.
/^(-?)(0x|@)/	В начале записи найти необязательный минус и префикс; $1 в замене сохраняет знак.
/^(-?)0(?=[0-7])/	Удалить ведущий восьмеричный префикс перед следующей цифрой, сохраняя знак. (?=...) — проверка вперёд без поглощения символов.

Не путайте ^ внутри строки SendKeys (^c — Ctrl+C) и ^ в регулярном выражении (начало строки). Значение символа зависит от языка и контекста.

13. Системы счисления с нуля

Основание b определяет набор цифр и вес позиции. В двоичной системе цифры 0 и 1; в восьмеричной 0…7; в десятичной 0…9; в шестнадцатеричной 0…9 и A…F, где A=10, B=11, C=12, D=13, E=14, F=15.

Запись входа	Основание	Значение DEC
0xF7	16	15 × 16 + 7 = 247.
010	8	1 × 8 + 0 = 8.
@101	2	1 × 4 + 0 × 2 + 1 = 5.
10	10	Десять.
0.5	10	Одна вторая; после начального нуля стоит точка.
010.4	8	8 + 4/8 = 8.5.

Входные префиксы — соглашение этой программы. Символ @ не является универсальным стандартом двоичных чисел во всех языках. Из записи 010 в текстовом файле основание определяет наш парсер, а не автоматический разбор числового литерала движком JScript.

Для перевода целого из b в DEC умножайте накопленное значение на b и добавляйте цифру. Для @101: 0×2+1=1, 1×2+0=2, 2×2+1=5. Для дроби вес первой цифры — 1/b, второй — $1/b^2$ и так далее.

Для перевода целого из DEC в b делите на b и записывайте остатки в обратном порядке. 374/16 = 23 остаток 6, 23/16 = 1 остаток 7, 1/16 = 0 остаток 1; ответ 176_{16}.

Для перевода дроби из DEC в b умножайте её на b; целая часть произведения — очередная цифра. 0.125×16=2.0, значит $0.125_{10} = 0.2_{16}$. Поэтому $4.125_{10} = 4.2_{16} = 4.1_{8} = 100.001_{2}$.

Основание входных чисел и основание результата независимы. При Output base: 16 строка 9*6 = 36 означает $54_{10} = 36_{16}$, а не ошибку умножения. Префиксы слева относятся к входу, заголовок — к ответам справа.

14. Зачем нужен парсер

Парсер преобразует строку в структуру, показывающую порядок вычисления. Отправить всю строку вроде (0x10+010)*@10 обычному Калькулятору нельзя: смесь префиксов — формат нашей программы. Сначала необходимо распознать числа и операции.

Три уровня: atom — число, знак или скобки; term — * и /; expr — + и −. expr вызывает term, а term — atom; поэтому умножение обрабатывается до сложения. При открывающей скобке atom вызывает expr повторно до закрывающей.

Для 2+3*4 сначала создаётся узел умножения 3 и 4, затем узел сложения с 2. evaluate сначала получает ответы детей, затем выполняет родительскую операцию. Порядок запросов: 3*4 → 12, 2+12 → 14. Для (2+3)*4: 2+3 → 5, 5*4 → 20.

Унарный минус относится к одному операнду: -5. Бинарный минус — к двум: 8-5. Рекурсия — вызов функцией самой себя прямо или через другие функции; здесь она нужна для вложенных скобок и обхода дерева.

Проверки парсера: допустимость цифр для основания, наличие операнда, закрытие скобок, отсутствие лишних символов после выражения. «08» не должно частично приниматься за 0. «2(3)» не является поддерживаемым неявным умножением: нужно 2*3.

15. Как устроен PowerShell-мост

JScript создаёт запрос в UTF-16LE и запускает calculator.ps1. Мост читает параметры, находит процесс Калькулятора, активирует его окно, очищает состояние, вводит операнды и операцию. Затем получает ответ с дисплея через UI Automation либо из буфера обмена, записывает OK и число в ответный файл. JScript ждёт завершения, проверяет код и читает число.

UI Automation — интерфейс доступности и автоматизации Windows. Через AutomationElement можно найти элемент, через ValuePattern или TextPattern — получить значение/текст, если приложение поддерживает такой шаблон. AutomationId — имя элемента интерфейса; это не PID и не HWND. В разных версиях Калькулятора идентификаторы могут различаться, поэтому такой способ нельзя объявлять универсальным.

Конструкция PowerShell	Что означает
param(...)	Объявление входных параметров скрипта.
$Request / $Response	Пути к запросу и ответу; знак $ обозначает переменную.
$ErrorActionPreference = 'Stop'	Перевод многих ошибок командлетов в прерывающие ошибки, обрабатываемые catch.
Add-Type -AssemblyName ...	Подключение сборки .NET для Forms/автоматизации.

Конструкция PowerShell	Что означает
New-Object -ComObject WScript.Shell	Создание того же COM-объекта оболочки из PowerShell.
Get-Process -Name ...	Поиск процессов по известным именам. Отсутствующее имя не доказывает отсутствие окна другой реализации.
Where-Object { ... }	Фильтрация найденных объектов.
Start-Process calc.exe	Запуск Калькулятора.
Start-Sleep -Milliseconds 500	Задержка полсекунды.
[IO.File]::ReadAllLines(...)	Статический вызов .NET для чтения строк файла.
[Text.Encoding]::Unicode	UTF-16LE в используемом .NET API.
[double]::Parse(..., InvariantCulture)	Чтение десятичного числа в согласованном формате обмена.
try / catch / throw	Контроль ошибок и возврат ERROR вместо фиктивного результата.
exit 0 / exit 1	Успех / ошибка процесса моста.

Параметры запуска: -NoLogo убирает баннер, -NoProfile отключает пользовательские профили, -STA задаёт COM-apartment, -File выбирает сценарий, -Request/-Response — наши параметры. -ExecutionPolicy Bypass действует в дочернем процессе и не даёт прав администратора. Не следует ради лабораторной отключать защиту Windows или глобально менять политику.

Региональный формат: входные выражения используют точку; при наборе в Калькулятор десятичный разделитель приводится к текущей культуре Windows. Ответ возвращается в согласованном формате для Number(). Если язык приложения отличается от региональных настроек, чтение всё равно может требовать адаптации.

16. Ошибки: как искать причину по этапам

Признак	Что проверить первым
«Файл не найден», строка открытия common.vbs	Совпадает ли комплект файлов с версией кода; нужен ли ему common.vbs.
Калькулятор открылся, но startup timed out	По каким именам процессов выполняется поиск; поддерживается ли классический calc/win32calc. Переустановка не следует автоматически.
На дисплее 54, а в Блокноте искажённый текст	SendKeys для кириллицы, раскладку и фокус; надёжнее Unicode-файл.
Calculator did not copy a result to the clipboard	Этап чтения результата, активное окно, реализацию копирования, текущую версию calculator.ps1.
Ошибка только у 08	Недопустимая восьмеричная цифра 8.
Ошибка Missing closing parenthesis	Баланс круглых скобок во входной строке.
ERROR: Division by zero	Значение правого операнда деления, в том числе после вычисления скобок.
В HEX справа 36 вместо 54	Это нормальный перевод 54_{10} в 36_{16}.
При отмене виден старый text2.txt	Отмена не создаёт новый файл; смотрите дату и содержимое прошлого результата.
Выход содержит ERROR, но файл создан	Сам факт создания не доказывает успех; смотрите errors и код выхода.

Порядок диагностики: воспроизведите на одном простом примере 9*6; посмотрите дисплей; проверьте чтение ответа; затем форматирование и запись. Не меняйте одновременно вход, версию сценария и настройки Windows — иначе невозможно понять причину изменения.

17. Точность, усечение и честная проверка

JScript Number — double: около 15–17 значащих десятичных цифр, безопасное целое не больше 2^53−1. Это не означает, что любая дробь с 15 цифрами представляется точно. Например, 0.1 не имеет конечной двоичной записи.

Ограничение 12 дробных цифр в format — отдельное ограничение вывода. Маркер [truncated] означает остаток после усечения. Точность Калькулятора может быть выше, но преобразование его ответа в Number снова ограничивает точность.

Проверка на известном эталоне отличается от подстановки эталона. Если сценарий получил 54 из Калькулятора и сравнил с 54 — это проверка. Если он при любой ошибке просто написал 54 — это не выполнение требования.

Тесты парсера с имитацией Калькулятора показывают, что правильно разобраны строки и вызваны операции. Они не проверяют горячие клавиши, буфер обмена и реальные окна Windows. Скриншот конечного файла показывает его содержимое, но не доказывает, что все невидимые ветви программы тестировались.

18. Упражнения для самостоятельной работы

Упражнение 1. Без запуска объясните каждый аргумент sh.Run "calc.exe", 1, False. Что изменится при True?

Упражнение 2. Запишите команды cmd для перехода в папку лабораторной на другом диске и запуска script3.js через JScript. Объясните ключ /d.

Упражнение 3. Объясните различие между F7, {F7}, {+} и + в SendKeys. Почему строка F7+7F= некорректна для нужного буквального плюса?

Упражнение 4. Рассчитайте вручную 0x1A+2, 010+@10, 0xA.8+@1.1 и 010.4/2. Запишите ответы DEC и HEX.

Упражнение 5. Для 2+3*4 и (2+3)*4 нарисуйте дерево и перечислите обращения к Калькулятору.

Упражнение 6. Прочитайте цикл reader.ReadLine. Объясните, зачем отдельно хранятся lineNumber и failures и почему пустые строки пропускаются.

Упражнение 7. Подготовьте отдельную копию входного файла и проверьте строки ниже. Не заменяйте единственную копию успешных результатов: сначала сохраните её отдельно.

```
9*6
0xF7+0x7F
010+@10
0xA.8+@1.1
2+3*4
(2+3)*4
010.4/2
0xFF
```

Упражнение 8. Отдельно проверьте 1/0, 08, @102 и (2+3. Объясните, на каком этапе должна возникать ошибка. В текущей программе ошибка одной строки не должна мешать обработке следующих.

Упражнение 9. Выполните выбор HEX и DEC, сохранив выходы под разными именами. При желании дополнительно OCT/BIN. Это способ проверки всех вариантов, а не требование исходного PDF запускать ровно четыре раза.

Упражнение 10. Напишите короткий сценарий, который выводит свой полный путь и вопрос Popup, но ничего не запускает. Пример для самопроверки:

```
var sh = new ActiveXObject("WScript.Shell");
WScript.Echo(WScript.ScriptFullName);
var r = sh.Popup("Путь показан. Закончить?", 0, "Практика", 1 + 64);
WScript.Echo("Ответ: " + r);
```

19. Ответы к упражнениям

1. calc.exe — команда, 1 — обычное активное окно, False — продолжить сценарий сразу. True может заставить ждать закрытия Калькулятора.

2. Например: cd /d "D:\Lab4", затем cscript.exe //nologo //E:JScript script3.js. /d меняет также активный диск.

3. F7 — две цифры HEX-числа; {F7} — функциональная клавиша; {+} — буквальный плюс; + — модификатор Shift.

4. 0x1A+2 = 28_{10} = $1C_{16}$. 010+@10 = 10_{10} = A_{16}. 0xA.8+@1.1 = 12_{10} = C_{16}. 010.4/2 = 4.25_{10} = 4.4_{16}.

5. 2+3*4: корень +, справа узел *; запросы 3*4, затем 2+12. (2+3)*4: корень *, слева узел +; запросы 2+3, затем 5*4.

6. lineNumber соответствует физической строке входа и нужен в сообщении об ошибке. failures считает ошибочные выражения. Пустая строка не является выражением; при её пропуске нумерация строк всё равно продолжается.

7. В DEC: 54, 374, 10, 12, 14, 20, 4.25, 255. В HEX: 36, 176, A, C, E, 14, 4.4, FF. Заголовок вывода указывает основание ответов.

8. 1/0 — проверка деления в evaluate; 08 и @102 — проверка цифр в parse; (2+3 — проверка закрывающей скобки. Ошибки интерфейса Windows — другой класс, они не заменяют эти ожидаемые проверки.

9. Один выходной файл перезаписывается при каждом запуске, поэтому его копируют до следующего. Отмена может оставить старый результат.

10. WScript.ScriptFullName даёт путь; Popup 1+64 — OK/Отмена и информационный значок. Возвращаемый код 1 — OK, 2 — Отмена.

20. Вопросы на защите с короткими ответами

Вопрос	Ответ, который нужно уметь раскрыть
Что такое WSH?	Хост сценариев Windows, предоставляющий окружение для VBScript/JScript.
Чем wscript отличается от cscript?	Способом взаимодействия: окна против консоли; язык сценария может быть тем же.
Что создаёт CreateObject?	COM-объект по зарегистрированному имени; здесь оболочку или FSO.
Почему в VBScript нужен Set?	Для присваивания объектной ссылки.
Зачем Option Explicit?	Чтобы переменные объявлялись и опечатки не создавали новые имена молча.
Run и Exec — одно и то же?	Нет: Exec даёт объект процесса и потоки; Run задаёт запуск, стиль окна и ожидание.
Run возвращает PID?	Нет. PID доступен у объекта, например возвращённого Exec.
Что принимает AppActivate?	PID или строку заголовка; возвращает успех/неуспех.
Зачем задержки?	Интерфейс асинхронен, ему требуется время на реакцию.
Почему + в фигурных скобках?	В SendKeys + — Shift; {+} вводит знак сложения.
Почему не вводить всю строку в обычный Калькулятор?	Он не понимает смешанный пользовательский формат 0x, 0, @ и порядок нашего парсера.
Где выполняется арифметика третьего задания?	Бинарные операции — в Калькуляторе, порядок и преобразования — в JScript.
Зачем PowerShell?	Для связи с интерфейсом и чтения ответа, а не для замены арифметики Калькулятора.
Зачем дерево?	Оно явно сохраняет приоритет и вложенность операций.
Как задаётся приоритет?	Разделением atom → term → expr и рекурсией для скобок.
Что делает identity?	Передаёт одиночное значение в Калькулятор при отсутствии бинарных операций.
Как обрабатываются неверные строки?	try/catch, запись ERROR с номером, продолжение обработки, общий код 1.
Что означает Output base?	Основание записи ответов, а не единое основание всех входных чисел.
Почему 010 — восемь?	Парсер трактует ведущий ноль перед цифрой как OCT-префикс.
Почему используется обычный режим для дробей?	Режим программиста Калькулятора рассчитан на целочисленную работу.
Что означает [truncated]?	Дробная запись усечена после 12 цифр с ненулевым остатком.
Все ли результаты на скриншотах оформлены правильно?	Числа совпадают; в первых двух заданиях подписи искажены — это отдельный дефект.

Вопрос	Ответ, который нужно уметь раскрыть
Нужно ли переходить с Windows 11 на Windows 10?	Нет; важнее версия Калькулятора и совместимость интерфейсной автоматизации.
Что доказывают тесты без Windows?	Логику парсера и преобразований, но не взаимодействие с реальными окнами.

21. План ответа на защите

Начните с цели: «Я автоматизирую стандартные приложения Windows и обрабатываю файл выражений». Затем назовите хост WSH и два основных COM-объекта. Для первого задания расскажите цепочку запуск → активация → ввод → получение ответа → Блокнот. Для второго объясните HEX/DEC и F7 против {F7}. Для третьего — файл, диалог, парсер, дерево, мост, результат.

Покажите один простой пример вручную: $0xF7+0x7F = 374_{10} = 176_{16}$. После этого объясните, почему 2+3*4 и (2+3)*4 различаются. Завершите обработкой ошибок и границами проверки: какие результаты видны, какие ветви запускались, где встречались проблемы интерфейса.

Не говорите «всё работает», если в выходе есть ERROR или подпись искажена. Лучше точно указать: вычисление верно, но была ошибка передачи текста; исправление меняет механизм вывода, а не математический ответ.

22. Контрольный список перед сдачей

Проверьте свои ФИО, группу и дату. Сопоставьте листинги с фактическими файлами: старый вариант через NoteKeys и новый через Unicode-файл нельзя описывать как одну неизменную версию. Убедитесь, что скриншоты читаемы, подписаны и содержат реальные результаты. Сохраните вход и выход каждой заявленной проверки. Умейте объяснить любую команду из своих листингов и назвать зависимость calculator.ps1. При запуске не отключайте защиту Windows и не выдавайте отсутствие ошибки хоста за успешное вычисление.

23. Где читать первоисточники

Названия страниц приведены для поиска в официальной документации Microsoft Learn. Старые разделы WSH могут находиться в архиве Previous Versions.

Microsoft Learn: Windows Script Host; WshShell Object; Run Method; Exec Method; AppActivate Method; SendKeys Method; Popup Method; FileSystemObject Object; OpenTextFile Method; CreateTextFile Method.

Microsoft Learn: about_PowerShell_exe; about_Execution_Policies; UI Automation Overview; AutomationElement Class. Для клавиш приложения — Microsoft Support, «Сочетания клавиш в приложениях Windows», раздел «Калькулятор».

При сравнении с найденными примерами проверяйте язык (VBScript, JScript или PowerShell), версию движка и то, относятся ли советы к классическому или современному Калькулятору. Не используйте первый попавшийся код, отключающий проверки или подставляющий ответы вместо работы с приложением.